Question 4:

(a) Process specification: specify the process, create process specification form (everything but process logic)

Overview of Process Specifications

A **process specification** (or minispec) is a detailed description of how a process transforms input data into output. It is created for **primitive processes** in a Data Flow Diagram (DFD) or for higher-level processes that decompose into child diagrams. In object-oriented design, process specifications can also describe **class methods** or **steps in a use case**.

The main purpose of process specifications is to capture the **decision-making logic and formulas** so that every output can be traced to its input(s). This reduces ambiguity, provides a precise description of what is accomplished, and validates the system design.

Key Goals:

1. **Reduce ambiguity:** Understand the details of how a process works.
2. **Obtain a precise description:** Create documentation for programmers.
3. **Validate the system design:** Ensure inputs and outputs align with the DFD.

Processes that may not require specifications:

- Simple input/output operations (e.g., reading/writing data).
- Simple validation processes (already defined in the data dictionary).
- Processes using prewritten code or standard subprograms.

Choosing a Process

To select a process for specification, choose one that:

- Is **primitive** or cannot be decomposed further.
- Involves **decision-making** or calculations.
- Affects multiple outputs or downstream processes.
- Requires **clarity for implementation** or communication with users.

Example: “Determine Quantity Available” in an inventory system:

- Checks if an item is available for sale.
- Creates backorder records if needed.
- Calculates available quantity for orders.

Process Specification Form

Number 1.3

Name Determine Quantity Available

Description Determine if an item is available for sale. If it is not available, create a backordered item record.

Determine the quantity available.

Input Data Flow

Valid item from Process 1.2

Quantity on Hand from Item Record

Output Data Flow

Available Item (Item Number + Quantity Sold) to Processes 1.4 & 1.5

Backordered item to Inventory Control

Type of Process

☑ Online ☐ Batch ☐ Manual

Subprogram/Function Name

Det_Av_Quantity( )

Process Logic:

IF the Order Item Quantity is greater than Quantity on Hand

Then Move Order Item Quantity to Available Item Quantity

Move Order Item Number to Available Item Number

ELSE

Subtract Quantity on Hand from Order Item Quantity

giving Quantity Backordered

Move Quantity Backordered to Backordered Item Record

Move Item Number to Backordered Item Record

DO write Backordered Record

Move Quantity on Hand to Available Item Quantity

Move Order Item Number to Available Item Number

ENDIF

Refer to: Name:

☑ Structured English ☐ Decision Table ☐ Decision Tree

Unresolved Issues: Should the amount that is on order for this item be taken into account? Would this, combined with the expected arrival date of goods on order, change how the quantity available is calculated?

Important to note -

Name - verb, short

Input data flow - all inputs required for process

Output data flow - all outputs generated

Online/Batch/Manual → Done by people without computers

- Online: process runs immediately when data entered
- Batch: Periodically runs on a group of data e.g. monthly invoice generating of customers

Refer to → reference related processes, forms etc.

Unresolved issues - ambiguities of data
decisions needing more clarification

(b) Process logic (for the particular process given, which logic to choose)

Structured English

When process logic involves formulas or iteration, or when structured decisions are not complex, an appropriate technique for analyzing the decision process is the use of structured English. As the name implies, structured English is based on (1) structured logic, or instructions organized into nested and grouped procedures, and (2) simple English statements such as add, multiply, and move. A word problem can be transformed into structured English by putting the decision rules into their proper sequence and using the convention of IF-THEN-ELSE statements throughout.

Writing Structured English

To write structured English, you may want to use the following conventions:

1. Express all logic in terms of one of these four types: sequential structures, decision structures, case structures, or iterations (see Figure 9.3 for examples).
2. Use and capitalize accepted keywords such as IF, THEN, ELSE, DO, DO WHILE, DO UNTIL, and PERFORM.
3. Indent blocks of statements to show their hierarchy (nesting) clearly.
4. When words or phrases have been defined in a data dictionary (as in Chapter 8), underline those words or phrases to signify that they have a specialized, reserved meaning.
5. Be careful when using "and" and "or," and avoid confusion when distinguishing between "greater than" and "greater than or equal to" and similar relationships. "A and B" means both A and B; "A or B" means either A or B, but not both. Clarify the logical statements now rather than waiting until the program coding stage.

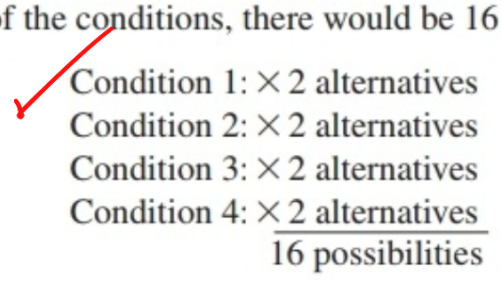
Structured English Type	Example
Sequential Structure A block of instructions in which no branching occurs	Action #1 Action #2 Action #3
Decision Structure Only IF a condition is true, complete the following statements; otherwise, jump to the ELSE	IF Condition A is True THEN implement Action A ELSE implement Action B ENDIF
Case Structure A special type of decision structure in which the cases are mutually exclusive (if one occurs, the others cannot)	IF Case #1 Implement Action #1 ELSE IF Case #2 Implement Action #2 ELSE IF Case #3 Implement Action #3 ELSE IF Case #4 Implement Action #4 ELSE print error ENDIF
Iteration Blocks of statements that are repeated until done	DO WHILE there are customers. Action #1 ENDDO

```
DO WHILE there are claims remaining
    IF claiment has not sent in a claim
        THEN set up new claimant record
    ELSE continue
    ENDIF

    Add claim to YTD Claim
    IF claiment has policy-plan A
        THEN IF deductible of $100.00 has not been met
            THEN subtract deductible-not-met from claim
            Update deductible
        ELSE continue
        ENDIF
        Subtract copayment of 40% of claim from claim
    ELSE IF claiment has policy-plan B
        THEN IF deductible of $50.00 has not been met
            THEN subtract deductible-not-met from claim
            Update deductible
        ELSE continue
        ENDIF
        Subtract copayment of 60% of claim from claim
    ELSE write plan-error-message
    ENDIF

    IF claim is greater than zero
        THEN print check
    ENDIF
    Print summary for claimant
    Update accounts
ENDDO
```

Developing Decision Tables

To build decision tables, an analyst needs to determine the maximum size of the table; eliminate any impossible situations, inconsistencies, or redundancies; and simplify the table as much as possible. The following steps provide the analyst with a systematic method for developing decision tables:

1. Determine the number of conditions that may affect the decision. Combine rows that overlap, such as conditions that are mutually exclusive. The number of conditions becomes the number of rows in the top half of the decision table.
2. Determine the number of possible actions that can be taken. That number becomes the number of rows in the lower half of the decision table.
3. Determine the number of condition alternatives for each condition. In the simplest form of a decision table, there would be two alternatives (Y or N) for each condition. In an extended-entry table, there may be many alternatives for each condition. Make sure that all possible values for the condition are included. For example, if a problem statement calculating a customer discount mentions one range of values for an order total from $100 to $1,000 and another range for greater than $1,000, the analyst should realize that the range from 0 up to $100 should also be added as a condition. This is especially true when there are other conditions that may apply to the 0 up to $100 order total.
4. Calculate the maximum number of columns in the decision table by multiplying the number of alternatives for each condition. If there were four conditions and two alternatives (Y or N) for each of the conditions, there would be 16 possibilities, as follows:



Condition 1: × 2 alternatives
Condition 2: × 2 alternatives
Condition 3: × 2 alternatives
Condition 4: × 2 alternatives
16 possibilities

5. Fill in the condition alternatives. Start with the first condition and divide the number of columns by the number of alternatives for that condition. In the foregoing example, there are 16 columns and two alternatives (Y or N), so 16 divided by 2 is 8. Then choose one

Conditions and Actions	Rules
Conditions	Condition Alternatives
Actions	Action Entries

Conditions and Actions	Rules 1	2	3	4
Under $50	Y	Y	N	N
Pays by check with two forms of ID	Y	N	Y	N
Uses credit card	N	Y	N	Y
Complete the sale after verifying signature.	X			
Complete the sale. No signature needed.		X		
Call supervisor for approval.			X	
Communicate electronically with bank for credit card authorization.				X

Conditions and Actions	Rules 1	2	3	4	5	6	7	8
Customer ordered from Fall catalog.	Y	Y	Y	Y	N	N	N	N
Customer ordered from Christmas catalog.	Y	Y	N	N	Y	Y	N	N
Customer ordered from specialty catalog.	Y	N	Y	N	Y	N	Y	N
Send out this year's Christmas catalog.		X		X		X		X
Send out specialty catalog.			X				X	
Send out both catalogs.	X				X			

can be expressed as:

Condition 1:	Y
Condition 2:	—
Action 1:	X

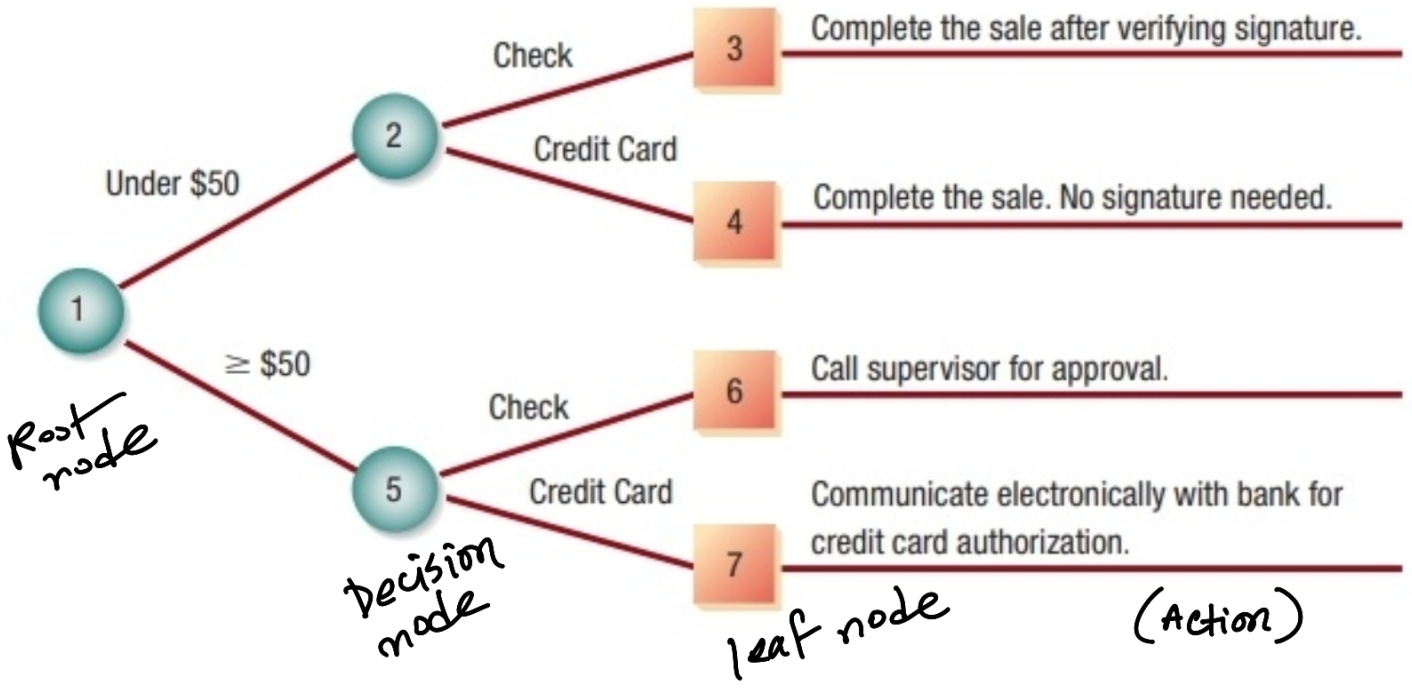
The dash [—] signifies that Condition 2 can be either Y or N, and the action will still be taken.

8. Check the table for any impossible situations, contradictions, and redundancies. They are discussed in more detail later.
9. Rearrange the conditions and actions (or even rules) if it makes the decision table more understandable.

modified

Conditions and Actions	Rules 1'	2'	3'	4'
Customer ordered from Fall catalog.	—	—	—	—
Customer ordered from Christmas catalog.	Y	—	N	—
Customer ordered from specialty catalog.	Y	N	Y	—
Customer ordered $50 or more.	Y	Y	Y	N
Send out this year's Christmas catalog.		X		
Send out specialty catalog.			X	
Send out both catalogs.	X			
Do not send out any catalog.				X

Decision tree – complex branching, sequence matters



Choosing a Structured Decision Analysis Technique

We have examined the three techniques for analysis of structured decisions: structured English, decision tables, and decision trees. Although they need not be used exclusively, it is customary to choose one analysis technique for a decision rather than employing all three. The following guidelines provide you with a way to choose one of the three techniques for a particular case:

1. Use structured English when
   a. There are many repetitious actions,

      OR

   b. Communication to end users is important.
2. Use decision tables when
   a. Complex combinations of conditions, actions, and rules are found,

      OR

   b. You require a method that effectively avoids impossible situations, redundancies, and contradictions.
3. Use decision trees when
   a. The sequence of conditions and actions is critical,

      OR

   b. When not every condition is relevant to every action (the branches are different).